

Proyecto de Software

MVC, Blueprints y templates

De una aplicación que funciona a una aplicación que podemos explicar.

Recorrido de la clase

1. Crecimiento y responsabilidades.
2. MVC como modelo para pensar.
3. Application Factory y Blueprints.
4. Templates y formularios.
5. Revisión de código generado por agentes.

Recuperamos Flask. No necesitamos haber implementado una base de datos.

El punto de partida

```
request → ruta → función → response
```

En Flask vimos rutas para:

```
GET    /solicitudes  
GET    /solicitudes/nueva  
POST   /solicitudes
```

¿Qué pasa cuando además aparecen reglas, datos y presentación?

Una función que hace demasiado

```
@app.post("/solicitudes")
def crear():
    # leer el formulario
    # decidir si la cantidad es válida
    # ejecutar SQL
    # armar HTML con concatenaciones
    # enviar una notificación
    # devolver la respuesta
    ...
```

Puede funcionar. ¿Qué necesitamos tocar para cambiar sólo la presentación?

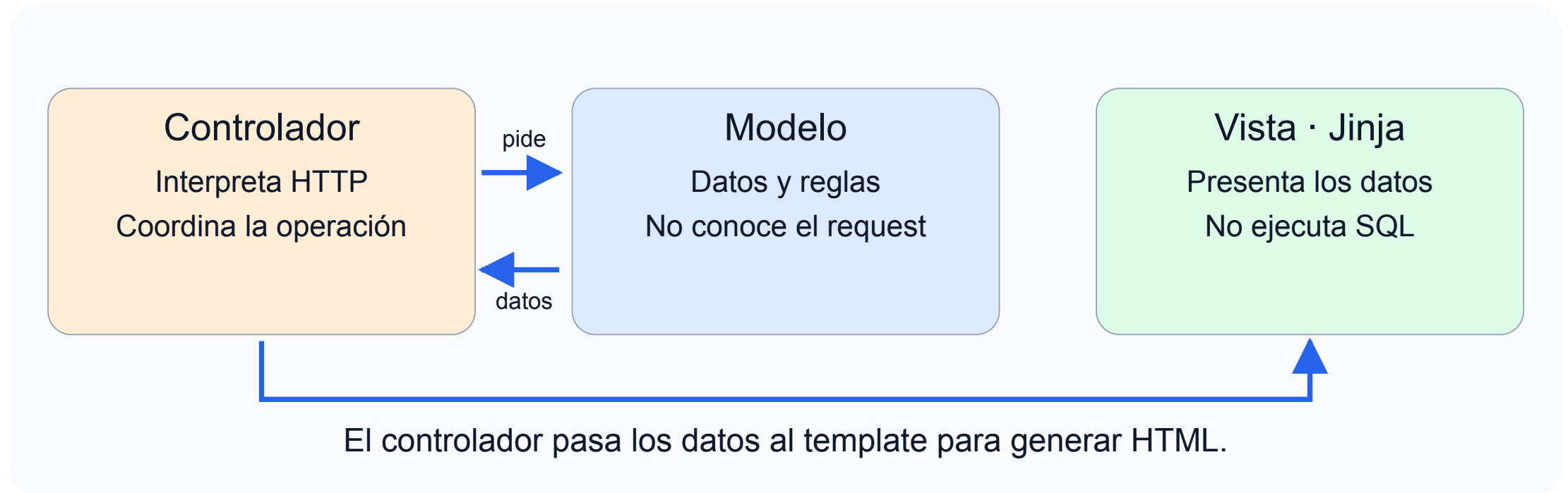
Separar por responsabilidad

Preguntas distintas:

- ¿Cómo interpreto este request?
- ¿Qué operación y reglas corresponden?
- ¿Cómo obtengo o guardo los datos?
- ¿Cómo presento el resultado?

Separar archivos ayuda sólo si también separa responsabilidades.

MVC: un mapa conceptual



No es una receta obligatoria de tres carpetas.

Flask no impone MVC: lo usamos para razonar sobre la aplicación.

El modelo no es sólo una tabla

Puede incluir:

- reglas del dominio;
- operaciones sobre solicitudes;
- colaboración con componentes de persistencia.

No debería depender de `request.form` ni producir HTML.

Una regla como “la cantidad debe ser positiva” debe poder probarse sin un navegador.

El controlador no hace todo

Su trabajo en este ejemplo:

1. Leer y convertir entradas HTTP.
2. Invocar una operación del modelo.
3. Traducir el resultado o error a una respuesta.

No concentra toda la lógica del negocio.

En aplicaciones mayores pueden aparecer servicios de aplicación. No necesitamos inventar todas esas capas hoy.

La vista presenta

El template puede:

- recorrer datos;
- mostrar un mensaje si no hay solicitudes;
- formatear y organizar el contenido.

No debería ejecutar SQL ni decidir quién puede autorizar una solicitud.

Ocultar un botón **no** protege la operación del servidor.

Atención con los nombres

Flask llama **view function** a la función asociada a una ruta.

En el mapa MVC de esta clase, esa función cumple principalmente el rol de **controlador**.

Jinja produce la presentación que llamamos **vista**.

■ No confundir el nombre usado por una API con la responsabilidad del código.

Checkpoint: ¿dónde lo ubicarías?

1. Leer `request.form["cantidad"]`.
2. Rechazar una cantidad no positiva.
3. Mostrar “No hay solicitudes”.
4. Consultar las solicitudes persistidas.
5. Responder con un error HTTP ante datos inválidos.

¿Alcanza con crear carpetas `models`, `views` y `controllers`?

Un ejemplo pequeño y explícito

Vamos a mostrar una lista de solicitudes y un formulario.

Para estudiar organización, el listado usa **datos fijos en memoria**.

En el POST validaremos la cantidad, pero **todavía no guardaremos una solicitud**.

Así no confundimos una demo de arquitectura con persistencia real ni con una solución del integrador.

Estructura de la demostración

```
proyecto/
├── src/
│   ├── __init__.py
│   ├── core/
│   │   ├── __init__.py
│   │   └── solicitudes.py
│   └── web/
│       ├── __init__.py
│       ├── controllers/
│       │   ├── __init__.py
│       │   └── solicitudes.py
│       ├── templates/
│       └── static/
└── pyproject.toml
```

Modelo: listar sin conocer Flask

src/core/solicitudes.py

```
def listar():  
    return [  
        {"id": 1, "servicio": "Servicio A", "cantidad": 20},  
        {"id": 2, "servicio": "Servicio B", "cantidad": 15},  
    ]  
  
def validar_cantidad(cantidad):  
    if cantidad <= 0:  
        raise ValueError("La cantidad debe ser positiva")
```

validar_cantidad recibe un entero, no un formulario.

Application Factory

src/web/__init__.py

```
from flask import Flask
from src.web.controllers.solicitudes import bp

def create_app(test_config=None):
    app = Flask(__name__)
    if test_config is not None:
        app.config.update(test_config)
    app.register_blueprint(bp)
    return app
```

Creamos y configuramos la aplicación en un lugar reconocible.

¿Qué nos permite la factory?



Permite crear instancias con configuraciones distintas, también para pruebas.

Desde la raíz del proyecto, con Flask instalado:

```
poetry run flask --app src.web:create_app run --debug
```

`--debug` sólo durante desarrollo. No incorporamos secretos fijos en el código.

Blueprint: agrupar operaciones web

Inicio de `src/web/controllers/solicitudes.py`:

```
from flask import Blueprint, render_template, request
from src.core import solicitudes

bp = Blueprint("solicitudes", __name__, url_prefix="/solicitudes")

@bp.get("")
def index():
    return render_template(
        "solicitudes/index.html", solicitudes=solicitudes.listar()
    )
```

Un Blueprint se registra en una aplicación; no es otra aplicación Flask.

URL y endpoint no son lo mismo

```
bp = Blueprint("solicitudes", __name__, url_prefix="/solicitudes")

@bp.get("")
def index(): ...
```

Concepto	Valor
URL	/solicitudes
Endpoint	solicitudes.index
Función Python	index

Usamos la ruta vacía para conservar `/solicitudes` sin barra final.

Checkpoint: el prefijo duplicado

Un agente agrega al mismo Blueprint:

```
@bp.get("/solicitudes/nueva")  
def nueva(): ...
```

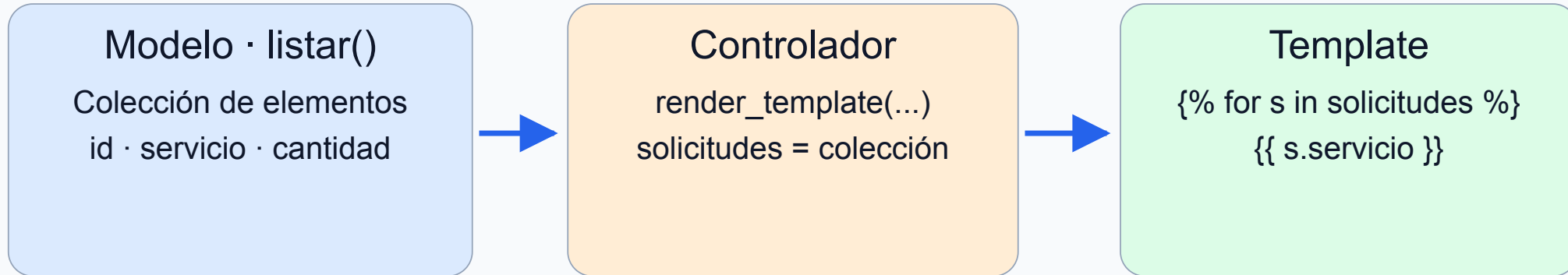
¿Qué URL queda registrada?

¿Cómo la escribiríamos para obtener `/solicitudes/nueva` ?

Revisar rutas efectivas, no sólo nombres de funciones.

El contrato con el template

```
return render_template(  
    "solicitudes/index.html",  
    solicitudes=solicitudes.listar(),  
)
```



Si cambian los nombres o la estructura, hay que adaptar el contrato.

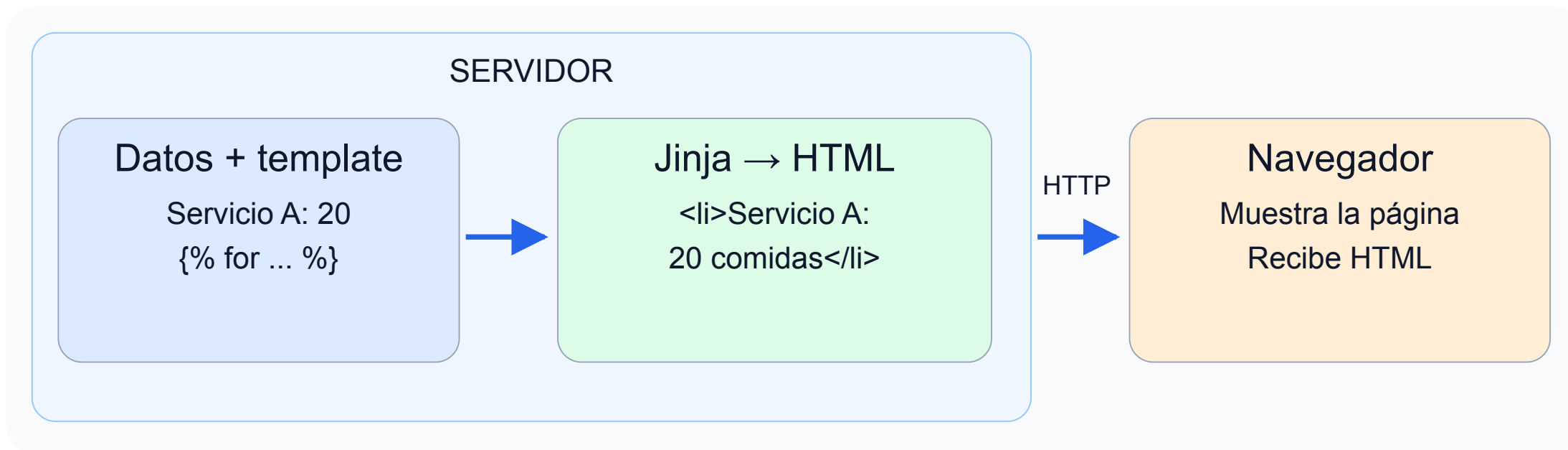
Primer template: una lista

src/web/templates/solicitudes/index.html — fragmento inicial:

```
<h1>Solicitudes</h1>
<ul>
  {% for solicitud in solicitudes %}
    <li>
      {{ solicitud.servicio }}: {{ solicitud.cantidad }} comidas
    </li>
  {% else %}
    <li>No hay solicitudes.</li>
  {% endfor %}
</ul>
```

{{ ... }} muestra valores; {% ... %} controla la presentación.

¿Dónde se ejecuta Jinja?



El navegador recibe HTML, no ejecuta el `for` de Jinja.

Eso es distinto de actualizar el DOM con JavaScript, que corresponde al recorrido frontend.

Un layout compartido

src/web/templates/layout.html

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>{% block title %}Solicitudes{% endblock %}</title>
  <link rel="stylesheet"
        href="{{ url_for('static', filename='app.css') }}">
</head>
<body>
  <main>{% block content %}{% endblock %}</main>
</body>
</html>
```

Heredar sin copiar toda la página

Versión final de `solicitudes/index.html` : envolver la lista anterior.

```
{% extends "layout.html" %}
{% block title %}Listado de solicitudes{% endblock %}
{% block content %}
    <h1>Solicitudes</h1>
    <!-- Aquí va la lista con for/else de la diapositiva anterior. -->
    <a href="{{ url_for('solicitudes.nueva') }}">Nueva solicitud</a>
{% endblock %}
```

`extends` reutiliza la estructura; cada `block` define un punto de extensión.

Archivos estáticos y enlaces

En `src/web/static/app.css` :

```
body { font-family: sans-serif; margin: 2rem; }  
.error { color: #a31515; }
```

```
<a href="{{ url_for('solicitudes.index') }}">Ver solicitudes</a>
```

`url_for` construye URLs a partir de endpoints.

No pegamos rutas del disco como `/home/docente/proyecto/...`.

Mostrar el formulario

En el mismo archivo de controladores:

```
@bp.get("/nueva")  
def nueva():  
    return render_template("solicitudes/nueva.html")
```

GET /solicitudes/nueva → formulario vacío

Mostrar la interfaz no crea una solicitud.

Formulario: enviar datos

solicitudes/nueva.html , dentro de su bloque content :

```
<h1>Nueva solicitud – demostración</h1>
<form action="{{ url_for('solicitudes.crear') }}" method="post">
  <label for="cantidad">Cantidad de comidas</label>
  <input id="cantidad" name="cantidad" type="number"
        min="1" required value="{{ cantidad | default('') }}">
  <button>Validar cantidad</button>
</form>
{% if error %}<p class="error" role="alert">{{ error }}</p>{% endif %}
{% if mensaje %}<p>{{ mensaje }}</p>{% endif %}
```

Hereda layout.html .

Recibir y validar sin fingir persistencia

En el mismo controlador, inicio de `crear` :

```
@bp.post("")
def crear():
    texto = request.form.get("cantidad", "")
    try:
        cantidad = int(texto)
        solicitudes.validar_cantidad(cantidad)
    except ValueError:
        return render_template(
            "solicitudes/nueva.html", cantidad=texto,
            error="Ingresa una cantidad entera positiva.",
        ), 400
```

El navegador ayuda; el servidor vuelve a validar.

Completar la respuesta de la demo

Continuación de `crear`, después del `except`:

```
return render_template(  
    "solicitudes/nueva.html",  
    cantidad=cantidad,  
    mensaje="Cantidad válida. Demo: no se guardó una solicitud.",  
)
```

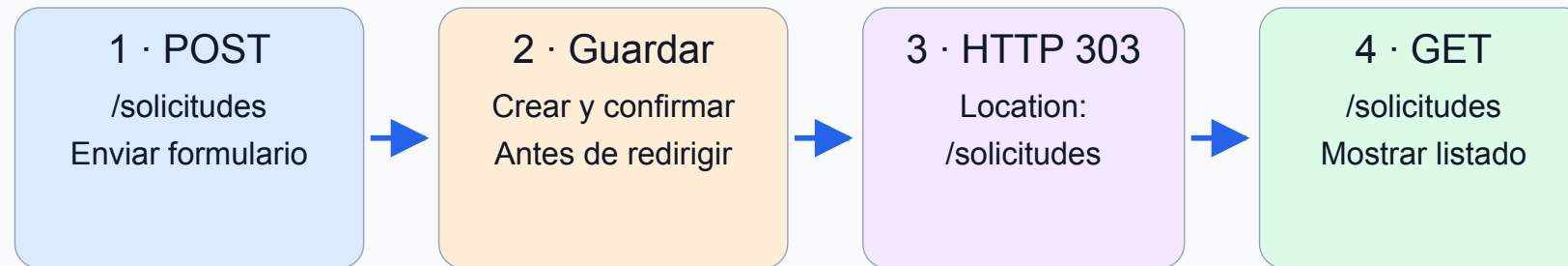
Hoy observamos quién interpreta, quién valida y quién presenta.

En BBDD y ORM incorporaremos persistencia; el mensaje de éxito debe corresponder a lo que realmente ocurrió.

Cuando sí exista creación: POST / Redirect / GET

Después de **guardar y confirmar** una solicitud:

```
return redirect(url_for("solicitudes.index"), code=303)
```



Al refrescar la página final se repite el GET, no el POST.

Importar `redirect`. Sólo después de una creación real.

PRG no garantiza por sí solo que nunca haya duplicados.

Checkpoint: una validación que se puede saltar

El agente propone:

“Con `min="1"` y `required` ya no hace falta validar en Python”.

¿Podemos enviar un POST sin usar ese formulario?

¿Qué debería ocurrir con `cantidad=abc`, `cantidad=0` o sin el campo?

Distinguir: ayuda de interfaz, validación de servidor y restricciones de persistencia.

Escape de datos en templates

```
<p>{{ observacion }}</p>
```

En los templates HTML habituales renderizados por Flask, Jinja escapa los valores automáticamente.

No agregar `|safe` a contenido aportado por usuarios para “arreglar cómo se ve”.

El escape no sustituye permisos, CSRF ni todas las defensas de seguridad. Es una protección específica del contexto de salida.

Diagnosticar el recorrido

Problema	Revisar primero
404 en una ruta esperada	Registro del Blueprint y URL efectiva
405	Método HTTP permitido
<code>TemplateNotFound</code>	Carpeta y nombre del template
<code>BuildError</code>	Endpoint y argumentos de <code>url_for</code>
Lista vacía inesperada	Datos enviados y contrato con el template

No mover archivos al azar: seguir `request → controlador → modelo → vista`.

¿Podemos comprobar la separación?

Sin Flask ni navegador:

```
from src.core.solicitudes import validar_cantidad

validar_cantidad(20) # No debe fallar.
validar_cantidad(0) # Debe lanzar ValueError.
```

Con un cliente de prueba HTTP comprobamos rutas y respuestas.

Si para probar una regla simple necesitamos renderizar toda la página, conviene revisar las dependencias.

Revisar una refactorización de un agente

Pedido: “Separá esta aplicación con Blueprints y templates”.

No alcanza con mirar el árbol de carpetas. Pedir evidencias:

- las URLs y los métodos siguen siendo los esperados;
- los templates reciben los mismos datos;
- las entradas inválidas se rechazan;
- las reglas no quedaron escondidas en HTML;
- no aparecieron secretos ni `|safe` innecesarios;
- el equipo puede recorrer una operación completa.

Checkpoint: dos diseños

A

El controlador valida reglas, consulta SQL, concatena HTML y devuelve todo. Hay cinco carpetas en el proyecto.

B

El controlador interpreta HTTP, invoca una operación independiente y pasa datos a un template. Hay tres carpetas.

¿Cuál separa mejor las responsabilidades? ¿Qué pruebas pedirían?

■ Más archivos y más abstracciones no implican mejor diseño.

Un cambio para pensar

Mañana queremos mostrar las mismas solicitudes como JSON.

- ¿Podemos reutilizar la operación de listado?
- ¿Qué parte de la presentación cambia?
- ¿Qué reglas deben seguir ejecutándose?

No necesitamos implementar la API hoy.

Esta pregunta conecta nuestra organización con la próxima clase de REST.

Lo importante de hoy

- MVC ayuda a discutir responsabilidades, no nombres de carpetas.
- Factory y Blueprints organizan la aplicación Flask.
- Jinja genera presentación en el servidor.
- El controlador adapta HTTP; el modelo no depende del formulario.
- Validar y probar sigue siendo necesario si el código lo escribió un agente.

Próximo problema: reemplazar datos de demostración por persistencia real.

Referencias

- [Flask: Application Factories](#)
- [Flask: Blueprints](#)
- [Flask: templates](#)
- [Jinja: herencia de templates](#)
- [Flask: testing](#)
- [Flask: consideraciones de seguridad](#)

Fin

Poder explicar el diseño también es parte de construirlo.